Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерных систем и сетей

Кафедра электронных вычислительных машин

Дисциплина: Жизненный цикл разработки программного обеспечения

ОТЧЕТ
по лабораторной работы №4
«Парное программирование»
на тему

CHALLENGE APP

Студент М.Д. Гончаров,
Р.П. Амбросевич

Преподаватель О.М. Внук

Минск 2026

СОДЕРЖАНИЕ

1 Выбор стиля парного программирования 3
1.1 Существующие стили парного программирования 3
1.1 Действующие стили парного программирования в рамках проекта 3
2 Польза и ошибки парного программирования 4
2.1 Извлечённая польза 4
2.2 Ошибки, найденные в процессе парного программирования 4
3 Впечатления о парном программировании 6
3.1 Какую задачу реализовывали? 6
3.2 Какой стиль показался более приемлемым? Почему? 6
3.3 Что получилось? Положительные впечатления. 6
3.4 Что не получилось? Как можно это поправить? 6
3.5 Будете ли использовать ПП для других задач? Каких? 6

1 ВЫБОР СТИЛЯ ПАРНОГО ПРОГРАММИРОВАНИЯ

1.1 Существующие стили парного программирования

Существует несколько стилей парного программирования:

– Ведущий-ведомый. В этом случае на втором месте, после разработки часто стоит обучение менее опытного программиста, как правило, именно он непосредственно пишет код. Второй программист более опытный, он подсказывает, направляет, дает рекомендации;

– На равных. В этом случае работают два примерно одинаковых по опыту разработчиков, время от времени меняющихся местами;

– Водитель-штурман. В этом случае программисты выбирают разные роли. Один пишет код, разбирается в деталях, а второй занимается архитектурой кода, решением логических задач, рисует схемы;

– Пинг-понг. Один программист пишет тест, а второй – реализацию под него. После происходит смена ролей;

– Удаленное ПП. Единственным минусом такого стиля является то, что нельзя ткнуть пальцем в экран. А если серьезно, то в этом случае можно использовать разные инструменты: например, давать партнеру возможность одновременно с вами писать код или же только видеть ваш экран.

1.1 Действующие стили парного программирования в рамках проекта

В данном проекте мы используем два вида парного программирования (ПП):

– На Равных;

– Удалённое ПП.

Более актуальным видом стало Удалённое ПП из-за общей занятости программистов и совместного решения, что данный вариант будет удобен и эффективен для продвижения проекта.

Для совместной работы над проектом на удалённой основе мы используем окружение Visual Studio 2026, Cursor и платформу для общения Discord.

2 ПОЛЬЗА И ОШИБКИ ПАРНОГО ПРОГРАММИРОВАНИЯ

2.1 Бонусы от парного программирования

В ходе работы над проектом "Challenge App" мы выделили несколько ключевых преимуществ парного программирования, которые напрямую повлияли на качество кода и скорость решения задач:

1. Обмен знаниями и непрерывное ревью кода. Поскольку мы работали в режиме «На равных», у нас происходил постоянный обмен мнениями. В первую очередь мы занялись проектированием архитектуры и разделением бизнесс идей на отдельные независимые домены. Далее мы занялись тем, что разработали общий план разработки и развития проекта. После общего проектирование архитектуры мы занялись уже реализацией отдельных сервисов. Каждому досталась половина сервисов.

2. Повышенная концентрация и дисциплина. Работая удаленно, легко отвлечься на посторонние дела. Однако сеансы парного программирования через Visual Studio, Cursor и Discord держали нас обоих в тонусе. Осознание того, что твой напарник сейчас наблюдает за твоими действиями и ждет следующего шага, мотивировало оставаться сфокусированным на задаче и не переключаться на другие вкладки или мессенджеры.

3. Выработка единого стиля кодирования. В начале проекта у нас были небольшие расхождения в форматировании кода и именовании переменных. Постоянное взаимодействие и обсуждение «в прямом эфире» помогли нам быстро прийти к единому стандарту. Это сделало код более читаемым и однородным, без необходимости позже тратить время на рефакторинг.

4. Эффективное решение сложных проблем. Когда один из нас заходил в тупик, пытаясь решить какую-то сложную логическую задачу, второй мог посмотреть на проблему свежим взглядом и предложить альтернативный путь. Этот эффект «второй головы» неоднократно помогал нам экономить время и находить более изящные решения, чем те, к которым мы могли бы прийти поодиночке.

2.2 Ошибки, найденные в процессе парного программирования

Несмотря на очевидную пользу, мы столкнулись с несколькими трудностями, характерными для удаленного парного программирования:

1. «Молчаливый партнёр». Временами, особенно при решении рутинных задач, один из нас, наблюдая за работой другого, замолкал. В такие моменты терялся главный смысл парной работы – обсуждение. Мы решили эту проблему, договорившись постоянно проговаривать свои мысли вслух (метод "Утёнка"). "Водитель" (тот, кто пишет код) объяснял, что и почему он делает, а "штурман" задавал уточняющие вопросы.

2. Конфликт настроек окружения. Сеансы совместной работы в Visual Studio иногда вызывали мелкие неудобства. Например, у одного из нас была настроена одна цветовая схема и комбинации горячих клавиш, а у другого – совершенно иные. Когда управление клавиатурой переходило от одного к другому, возникала короткая заминка, требовавшая адаптации. Это не критическая ошибка, но она немного сбивала с ритма.

3. Микро-отвлечения. Хоть общая концентрация и повысилась, иногда возникала проблема, когда один партнёр был полностью погружён в код, а второй в этот момент проверял уведомление в телефоне или отвечал на сообщение. Это приводило к тому, что приходилось повторять вопрос или объяснение. Мы минимизировали эту проблему, договорившись о коротких перерывах каждые 45-60 минут, чтобы каждый мог решить свои мелкие дела.

3 ВПЕЧАТЛЕНИЯ О ПАРНОМ ПРОГРАММИРОВАНИИ

3.1 Какую задачу реализовывали?

Мы работали над проектом «Приложение челенджей для игр».

3.2 Какой стиль показался более приемлемым? Почему?

Основной используемый стиль был “На Равных”. Так как оба разработчика имеют удовлетворительный уровень навыков и знаний.

Также мы использовали пинг-понг. Один выставлял требования для другого, а тот их реализовывал; и обратно.

3.3 Что получилось? Положительные впечатления.

– Так как проектированием системы и архитектуры занимались оба, оба имеют полное понимание того, что есть и как что работает.

– В следствии того, что задачи были разделены, общий процесс разработки шел быстрее и менее напряжно.

– Обмен опытом. Так как у нас разный стек, и от части разные уровни углубленности в определенные темы, был произведен профессиональный обмен опытом.

– Больше общения. До этого мы с коллегой не общались, но тут пришлось.

– Кооперативный час критического мышления. В моментах мы друг другу накидывали идеи, и так вышло что мы придумывали лучшие решения.

– Улучшение дисциплины. Так как нас двое, и один зависит от второго, приходится более усерднее работать, чтобы оба могли работать комфортно.

3.4 Что не получилось? Как можно это поправить?

– Мусорный обмен опытом. Иногда мы обменивались излишне не нужными деталями.

– Кооперативный брейншторм. Не всегда второй считает, что другой придумал лучшее решение.

– Дедлайны. Не всегда получалось синхронизоваться и успевать все реализовать в сроки.

– Рассинхронизация. Рабочее окружение, не валидные версии копий репозитория иногда приводят к тому, что в репозитории появляются конфликты.

3.5 Будете ли использовать ПП для других задач? Каких?

Да, мы планируем использовать парное программирование для решения сложных архитектурных задач, отладки нетривиальных багов и освоения

новых технологий. Этот подход максимально эффективен там, где требуется глубокий анализ и совместное принятие решений.